```
!pip install gensim
import gensim
import gensim.downloader as api
from gensim.models import KeyedVectors
import numpy as np
import pandas as pd
import nltk
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

```
lecting gensim
ownloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 k
uirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim)
uirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (
uirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gens
uirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.
nloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 48.6 MB/s eta 0:00:00
talling collected packages: gensim
cessfully installed gensim-4.4.0
```

```
import gensim.downloader as api

# Load Google News Word2Vec
word2vec_model = api.load("word2vec-google-news-300")

print("Vocabulary Size:", len(word2vec_model))
print("Vector Dimension:", word2vec_model["king"].shape)
print("Vector for 'king':", word2vec_model["king"][:10])
```

```
Vocabulary Size: 3000000
Vector Dimension: (300,)
Vector for 'king': [ 0.12597656  0.02978516  0.00860596  0.13964844 -0.02563477 -0.03613281
  0.11181641 -0.19824219  0.05126953  0.36328125]
```

```
!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip -q glove.6B.zip

from gensim.models import KeyedVectors

glove_model = KeyedVectors.load_word2vec_format(
    "glove.6B.50d.txt",
    binary=False,
    no_header=True
)

print("Vocabulary Size:", len(glove_model))
print("Vector Dimension:", glove_model["king"].shape)
```

```
--2026-02-10 04:17:32--  http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-02-10 04:17:32--  https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
```

```
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-02-10 04:17:32--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: ‘glove.6B.zip’

glove.6B.zip        100%[===================>] 822.24M  5.00MB/s    in 2m 41s

2026-02-10 04:20:13 (5.10 MB/s) - ‘glove.6B.zip’ saved [862182613/862182613]

Vocabulary Size: 400000
Vector Dimension: (50,)
```

```
pairs = [
    ("doctor","nurse"),
    ("cat","dog"),
    ("car","bus"),
    ("king","queen"),
    ("teacher","student"),
    ("india","china"),
    ("computer","keyboard"),
    ("apple","mango"),
    ("sun","moon"),
    ("city","village")
]

for w1, w2 in pairs:
    print(w1, "-", w2, ":", word2vec_model.similarity(w1, w2))
```

```
doctor - nurse : 0.63195235
cat - dog : 0.76094574
car - bus : 0.4693371
king - queen : 0.6510957
teacher - student : 0.63013655
india - china : 0.35330796
computer - keyboard : 0.39639163
apple - mango : 0.57518554
sun - moon : 0.4262834
city - village : 0.47896868
```

```
words = ["king", "university", "hospital", "india", "computer"]

for w in words:
    print("\nNearest words to:", w)
    print(word2vec_model.most_similar(w, topn=5))
```

```
Nearest words to: king
[('kings', 0.7138045430183411), ('queen', 0.6510956883430481), ('monarch', 0.6413194537162781), ('

Nearest words to: university
[('universities', 0.7003918886184692), ('faculty', 0.6780907511711121), ('unversity', 0.6758289933

Nearest words to: hospital
[('Hospital', 0.7931912541389465), ('hopsital', 0.7784007787704468), ('hosptial', 0.75817799568176

Nearest words to: india
```

```
[('indian', 0.6967039704322815), ('usa', 0.6836211085319519), ('pakistan', 0.681516706943512), ('c

Nearest words to: computer
[('computers', 0.7979379892349243), ('laptop', 0.6640493273735046), ('laptop_computer', 0.65488684
```

```
print("king - man + woman =")
print(word2vec_model.most_similar(
    positive=["king","woman"],
    negative=["man"],
    topn=1
))

print("paris - france + india =")
print(word2vec_model.most_similar(
    positive=["india","paris"],
    negative=["france"],
    topn=1
))
```

```
king - man + woman =
[('queen', 0.7118193507194519)]
paris - france + india =
[('chennai', 0.5442505478858948)]
```

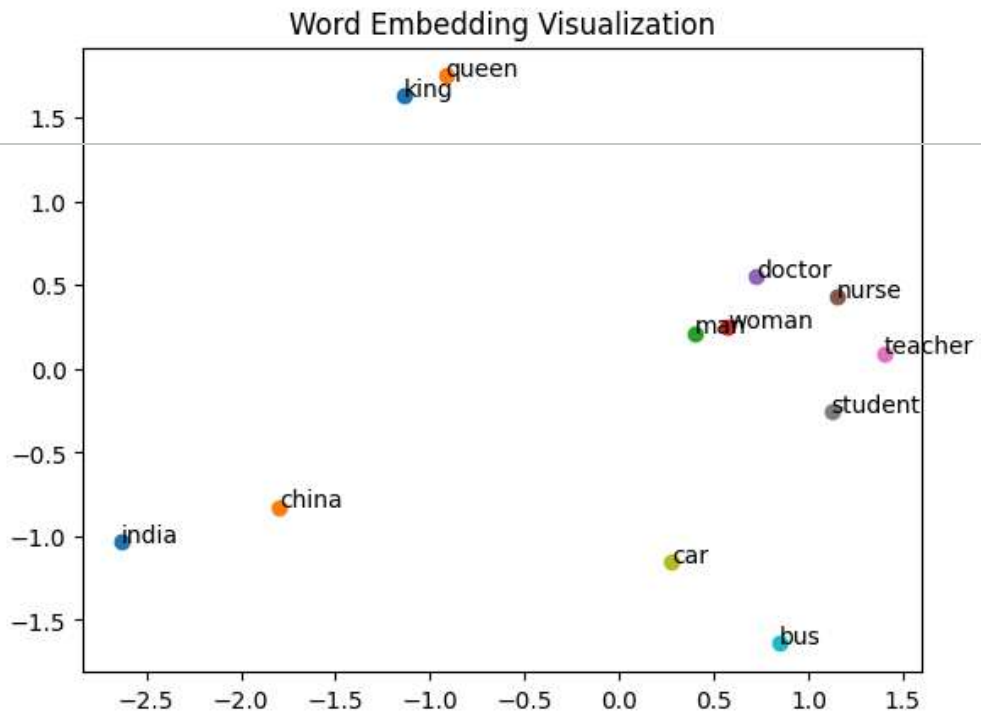
```
words = ["king","queen","man","woman","doctor","nurse",
         "teacher","student","car","bus","india","china"]

vectors = [word2vec_model[w] for w in words]

pca = PCA(n_components=2)
reduced = pca.fit_transform(vectors)

plt.figure()
for i, word in enumerate(words):
    plt.scatter(reduced[i,0], reduced[i,1])
    plt.text(reduced[i,0], reduced[i,1], word)

plt.title("Word Embedding Visualization")
plt.show()
```


Word Embedding Visualization
queen
king
doctor
nurse
man
woman
teacher
student
china
india
car
bus
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5